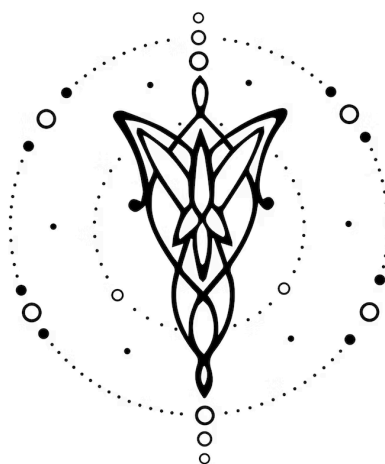




Informe

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

V1.0.0

1. Equipo: Calendar	1
2. Introducción	2
3. Modelo Computacional	2
4. Implementaciones	3
5. Futuras Extensiones	5
6. Conclusiones	6
7. Bibliografía	6

1. Equipo: Calendar

Nombre	Apellido	Legajo	E-mail
Alexis	Herrera Vegas	64045	aherreravegas@itba.edu.ar
Jeronimo	Esquivel	64756	tesquivel@itba.edu.ar
Andres	Cortese	64612	acortese@itba.edu.ar
Sebastian	Caules	64331	scaules@itba.edu.ar

2. Introducción

Este trabajo práctico, desarrollado para la materia Teoría de Lenguajes y Autómatas, consiste en el diseño e implementación de un compilador para un lenguaje específico de dominio (DSL) orientado a la planificación de un calendario. El lenguaje permite crear bloques de año y mes, y definir eventos puntuales o recurrentes durante el mes, con un horario específico y propiedades como color, descripción y enlace. El objetivo final es generar automáticamente un calendario completo en formato HTML con los eventos definidos.

El desarrollo se dividió en tres etapas: diseño del lenguaje, implementación del frontend (análisis léxico y sintáctico con Flex y Bison), y desarrollo del backend, encargado de validar semánticamente el programa y generar la salida final. Este informe describe cada una de esas etapas y el proceso seguido para construir la solución.

3. Modelo Computacional

3.1 Dominio

El objetivo del proyecto es desarrollar un lenguaje de programación específico (DSL) que permita definir, visualizar y manipular agendas de eventos en un calendario, ofreciendo una sintaxis expresiva centrada en el dominio del tiempo.

El lenguaje debe posibilitar la creación de calendarios mensuales o anuales, con soporte para eventos puntuales o recurrentes, y ofrecer herramientas para reemplazar, sobrescribir o extender eventos existentes mediante construcciones de alto nivel.

Cada evento define un instante o período temporal (por ejemplo, una clase semanal o una reunión de media hora), junto con atributos descriptivos como nombre, horario, color o enlace.

El resultado de la interpretación del lenguaje será un calendario HTML visualmente representativo, donde los eventos se renderizan con sus propiedades y colores.

El objetivo es facilitar la planificación y acelerar su proceso. Actualmente se utilizan herramientas como el papel o Excel que no están específicamente enfocadas en la creación de agendas, por lo que nuestro lenguaje busca facilitar esta última mediante una sintaxis legible, simple y natural para la definición de cronogramas.

3.2 Lenguaje

Adentrándonos más específicamente al lenguaje, como se mencionó anteriormente, se optó por una sintaxis declarativa, evitando estructuras más rígidas como las de estilo JSON. Esto con el objetivo de lograr un estilo claro y amigable, orientado a una redacción que resulte intuitiva.

El lenguaje está enfocado en describir de manera clara y concisa, todos los elementos y aspectos relevantes que surgen en la organización de horarios. En resumen, el lenguaje se centra simplemente en declarar que se quiere lograr.

Profundizando un poco más, el DSL permite definir las siguientes entidades principales:

- Header: Permite establecer un huso horario y definir colores personalizados para que puedan ser utilizados en la declaración de eventos.
- Bloques de año y meses: Mediante una estructura de bloques con llaves, familiar con otros lenguajes de programación, permite definir el año y los distintos meses cuando sucederán los eventos declarados.
- Eventos: Permite declarar eventos con un título identificador y un horario. Un evento puede ser puntual y suceder en un solo día del mes, o puede ser recurrente y suceder en uno o varios días de la semana durante todo el mes. Además, el lenguaje permite añadir información adicional a los eventos, como una descripción, un enlace o el color con el que se va a representar en el calendario final de formato HTML.
- Sobreescritura: Permite modificar un evento ya existente al cambiar su descripción, su enlace o su color.

4. Implementaciones

4.1 Frontend

Para esta parte se usaron Flex y Bison. Con Flex se armó el lexer que reconoce palabras clave (YEAR, MONTH, EVENT, etc.), identificadores, strings, números, horas, y demás elementos del lenguaje. Después con Bison se hizo el parser, que se encarga de ver que el programa tenga una estructura válida según nuestra gramática, y de ahí se construyó un AST (árbol de sintaxis abstracta). La estructura del AST es la siguiente:

- Un header que contiene, si es que existen, el huso horario especificado y las definiciones de colores personalizados.
- Un único bloque de año que contiene un vector de punteros a bloques de mes. Cada mes se guarda en su respectivo índice, por lo que no pueden haber dos declaraciones de un mismo mes.
- Bloques de mes que contienen cada uno una lista de declaraciones, que pueden ser eventos o sobreescrituras (*override*).

- La declaración puede ser un evento, en ese caso contiene un identificador, un cuerpo con su información adicional y las especificaciones de su ocurrencia en el tiempo. Estas últimas se componen de su horario, el día del mes en el que ocurren o si es un evento recurrente, un vector que contiene los días de la semana en el que ocurre.
- La declaración también puede ser una sobreescritura, en ese caso contiene el identificador del evento a sobrecribir y el nuevo cuerpo.

Si bien muchas validaciones (léxicas y sintácticas) se resolvieron directamente desde el parser gracias a la expresividad de Bison, hubo ciertos controles que, por la naturaleza del autómata de pila y sus restricciones estructurales, no se pudieron implementar en el frontend. Estas validaciones fueron delegadas al backend.

4.2 Backend

Con el AST ya creado y estructurado, el back se va a encargar de recorrer sus nodos para interpretar y discriminar correctamente las declaraciones hechas por el usuario en el programa fuente.

Se creó una tabla de símbolos en *SymbolTable.c* donde se van a guardar las diferentes variables y constantes de nuestro programa, lo que nos va a permitir realizar validaciones y evitar colisiones. Nuestro lenguaje cuenta con dos tipos de entidades declarables: colores y eventos. Es muy importante tener la tabla para ordenar y optimizar las validaciones necesarias.

Las validaciones se realizan en el *Validator.c*. Aquí se recorre el AST y se realizan las validaciones semánticas que delegó el frontend. Algunos ejemplos de estas son:

- Elección de un huso horario real.
- Eventos con fecha válida: año coherente (entre 0 y 9999) y mes existente (entre 1 y 12), día del mes acorde a la cantidad de días de ese mes (evitar incoherencias como 30 de febrero).
- Eventos con horarios coherentes: El horario de inicio no puede ser posterior al horario de fin (esto no permite eventos que ocurren durante más de un día).

Además, se realizan las siguientes validaciones mediante la tabla de símbolos:

- Los colores utilizados para eventos deben haber sido definidos previamente en el header.
- Los eventos no pueden tener un mismo identificador, ni superponerse temporalmente con otros eventos.
- Las declaraciones de sobreescritura deben sobrecribir un evento que ya se haya guardado en la tabla de símbolos.

Una vez validado el programa se pasa a la etapa de generación de código en *Generator.c*. Este programa se sirve del AST y de la tabla de símbolos para generar un archivo HTML que consiste en un calendario interactivo. El archivo HTML generado lleva el nombre de *index.html* y se genera en la raíz del proyecto.

4.3 Dificultades encontradas

A lo largo del desarrollo del trabajo práctico fuimos encontrando diferentes dificultades.

Desde un primer momento la comprensión del funcionamiento de lenguajes nuevos como Flex y Bison supuso un tiempo de aprendizaje, pero una vez comprendida la lógica y la relación entre los archivos del proyecto pudimos mantenernos sobre una misma línea con la lógica del proyecto base.

La confección del AST fue un gran desafío, ya que tuvimos que combinar varias estructuras de datos y ser cuidadosos a la hora de gestionar memoria y recorrerlos. Buscamos siempre elegir estructuras que se adapten correctamente a cada dato y permitan optimizar las operaciones. Un ejemplo de esto es la elección de vectores para conjuntos de tamaño conocido y elementos ya asociados a un índice (como meses y días de la semana) y listas enlazadas para conjuntos de tamaño desconocido y elementos más complejos (como Statements). En general fue un proceso interesante y sirvió para refrescar contenido de materias anteriores.

Estas mismas dificultades provocaron el descarte de ciertas funcionalidades propuestas inicialmente. A cierta altura del proyecto fue preferible solidificar la base y no volar muy cerca del Sol, con la posibilidad de tener que reestructurar todo en poco tiempo.

En cuanto a la devolución del Stage II, se corrigieron casi todos los errores o detalles marcados por la cátedra. Observaciones como warnings, uso del pipeline CI, permisos de ejecución de los scripts de bash, cantidad de tests, defaults en switch y backtracking fueron solucionadas. La única observación que no pudimos corregir es la independencia entre Flex y Bison para el ciclo de vida de ciertas variables, duplicadas con strdup. Se intentaron efectuar correcciones pero resultaron en problemas de leaking difíciles de solucionar prolijamente, por lo que decidimos no desperdiciar el tiempo en eso.

Agradecemos de corazón la lógica de debugging ya implementada en el proyecto base. Resolver errores resultó muy fácil gracias al seguimiento del logger. Tomamos nota para próximos proyectos.

5. Futuras Extensiones

A pesar de que el proyecto ya es funcional y cumple con su objetivo, se deja dilucidar que el lenguaje puede ser aún más potente y contar con más funcionalidades para futuras extensiones. Algunas de estas son:

- **Override:** Actualmente se puede sobrescribir únicamente la información adicional de un evento, por lo que un objetivo futuro será poder cambiar también información primordial de un evento como su título identificador o su horario, tanto para todos los eventos recurrentes como para una fecha puntual.

- Recurrencia en rangos específicos: Poder declarar eventos que ocurran únicamente entre ciertas fechas. Por ejemplo, todos los lunes del mes desde el día 1 hasta el día 15.
- Recurrencia bisemanal: Poder declarar eventos que ocurran cada dos semanas.
- Recurrencia sobre varios meses: Ahora cada evento recurrente está limitado al mes en el que fue declarado. Esto no parece óptimo para eventos que duren varios meses como puede ser una clase de la facultad.
- Eventos que duren varios días.
- Poder tener múltiples bloques de años en un mismo programa.
- Colores predeterminados ya listos para usar sin definir en el header: Puede mejorar la experiencia no tener que definir a mano colores básicos como rojo o azul.
- Eventos relacionados: Poder conectar varios eventos y que las modificaciones en uno afecten al resto.
- Mejorar el diseño del calendario final generado mediante el archivo HTML, ya que el diseño de este no fue una de las prioridades del proyecto.

6. Conclusiones

En conclusión, se desarrolló un trabajo que resuelve la problemática dispuesta inicialmente de forma satisfactoria. Se logró diseñar e implementar un compilador para un lenguaje de dominio específico (DSL) y se logró un lenguaje legible, simple y familiar para facilitar la creación de cronogramas y la planificación en el calendario.

Fue un trabajo desafiante y finalmente muy enriquecedor. Aprendimos nuevos lenguajes y comprendimos el funcionamiento de un compilador mediante la práctica, algo que parecía totalmente lejano a inicios del cuatrimestre.

7. Bibliografía

El material de la cátedra fue claro y de gran ayuda:

(2025-09-03, v1.0.0) Análisis Léxico :  (2025-09-03, v1.0.0) Análisis Léxico

(2024-05-08, v0.1.0) Análisis Sintáctico:  (2024-05-08, v0.1.0) Análisis Sintáctico

(2024-09-25, v0.2.0) Análisis Semántico:  (2024-09-25, v0.2.0) Análisis Semántico

(2024-05-16, v0.1.0) Generación de Código:  (2024-05-16, v0.1.0) Generación de Código